

WAB

Provadis School of International Management and Technology

**Analyzing the Correlation Between Cyclomatic
Complexity and Code Quality in Highly
Instrumental Python Projects**

Rubin Chempananickal James
rubin.chempananickal-james@stud-provadis-hochschule.de
Matriculation Number: D876

Department: Information Technology
Module: Agile Softwareentwicklung und Softwaretechnik
Reviewer: Philip Ceh

14.05.2026

Abstract

This study investigates whether cyclomatic complexity is associated with bug-proneness in important Python packages and how bug-fixing commits relate to later complexity change. Ten highly depended-upon PyPI packages were selected by reverse dependency rank and mined over a five-year window. The resulting dataset covers 47,151 functions. Cyclomatic complexity was measured with Lizard, bug-fixing commits were identified through a commit-message heuristic, and likely bug-introducing commits were approximated with a simplified SZZ procedure.

The results show a consistent but moderate positive relationship between function complexity and bug-fix activity across all ten repositories. Maintenance is also concentrated in more complex code: the top 10% most complex functions account for 29.1% of bug-fix commits, and the top 20% account for 47.3%. Most bug-fixing commits do not change measured complexity, and when complexity does change, increases are more common than decreases. The findings therefore support interpreting cyclomatic complexity as a useful indicator of elevated maintenance risk, but not as direct proof of future defects.

Abstrakt

Diese Studie untersucht, ob ein Zusammenhang zwischen zyklomatischer Komplexität und Fehleranfälligkeit in wichtigen Python-Paketen besteht und wie sich Bugfix-Commits auf die spätere Komplexität von Funktionen auswirken. Dafür wurden zehn stark abhängige PyPI-Pakete anhand ihrer Reverse-Dependency-Ränge ausgewählt und über ein Zeitfenster von fünf Jahren analysiert. Der resultierende Datensatz umfasst 47.151 Funktionen. Die zyklomatische Komplexität wurde mit Lizard gemessen, Bugfix-Commits wurden über eine Commit-Message-Heuristik identifiziert, und wahrscheinlich fehlerverursachende Commits wurden mit einem vereinfachten SZZ-Verfahren angenähert.

Die Ergebnisse zeigen über alle zehn Repositorien hinweg einen konsistenten, aber nur moderaten positiven Zusammenhang zwischen Funktionskomplexität und Bugfix-Aktivität. Wartungsaufwand konzentriert sich zudem auf komplexeren Code: Die komplexesten 10% der Funktionen vereinen 29,1% aller Bugfix-Commits auf sich, die komplexesten 20% bereits 47,3%. Die meisten Bugfix-Commits verändern die gemessene Komplexität nicht, und wenn sich die Komplexität verändert, sind Erhöhungen häufiger als Verringerungen. Zyklomatische Komplexität erweist sich damit als nützlicher Hinweis auf erhöhtes Wartungsrisiko, aber nicht als direkter Beleg für spätere Defekte.

Contents

Abstract	II
Abstrakt	II
List of Figures	IV
List of Tables	V
Glossary	VI
Abbreviations	VII
1. Introduction	1
1.1. Objectives	1
1.2. Related work	1
1.3. Research Question	1
2. Methods	2
2.1. Package Selection	2
2.2. Complexity Analysis	3
2.3. SZZ Attribution	3
3. Results	4
3.1. Dataset Overview	4
3.2. Complexity and Bug-fix Correlation	4
3.3. Maintenance Concentration	5
3.4. Complexity Changes in Bug-fixing Commits	7
3.5. SZZ Attributions	8
3.6. Time To Fix	9
4. Discussion	11
4.1. Limitations	11
5. Conclusion	13
5.1. Future Work	13
References	i
AI Declaration	ii
Declaration of Authorship	v

List of Figures

Figure 1	Share of functions touched by at least one bug-fix commit, and mean bug-fix commits per function, bucketed into groups of 20.	6
Figure 2	Cumulative concentration of bug-fix activity among the most complex functions.	7
Figure 3	Complexity-change categories observed at bug-fixing commits. No-change outcomes dominate, and increases are more common than decreases.	8
Figure 4	Complexity-change categories for simplified SZZ attributions. A substantial share of attributions cannot be recovered at function level, but matched rows still show more increases than decreases.	9
Figure 5	Distribution of time lags between attributed bug-introducing commits and their corresponding fixes.	10

List of Tables

Table 1	The selected packages and their number of analyzed functions.	2
Table 2	Packages with their reverse dependency rank, version, and number of analyzed functions.	4
Table 3	Correlation coefficients between function complexity and bug-fix commit frequency for the ten analyzed packages.	5
Table 4	Complexity bucket summary. The mean bug-fix count is computed over all functions in each bucket, including functions with zero bug-fix commits. ...	6

Glossary

SZZ, Sliwezki-Zeller-Zimmermann: The Sliwezki-Zeller-Zimmermann algorithm, a technique for identifying bug-introducing commits by tracing back from bug-fixing commits through version control history. (pp. 1, 3, 4, 8, 9, 10, 11, 12, 13)

cyclomatic complexity: A software metric used to measure the complexity of a program by counting the number of linearly independent paths through the program's source code. (pp. 1, 3, 9, 11, 12, 13)

Abbreviations

AWS , Amazon Web Services	(pp. 2)
LLM , Large Language Model	(pp. 1)
PyPI , Python Package Index	(pp. 1, 2)

1. Introduction

Python¹ is one of the most popular programming languages in the world², with a vast ecosystem of third-party packages available through the Python Package Index (PyPI)³. As the adoption of Python continues to grow, and especially as the frequency of commits to open-source packages by Large Language Models (LLMs) increases, understanding the quality and maintainability of packages, especially those “lynchpin” packages that other packages are built upon, becomes increasingly important for developers and organizations.

Cyclomatic complexity is a graph theory based approach to compute the number of independent paths through a function⁴. This gives a rough estimate of how many separate test cases one might need to cover every output of that function. A function with a higher cyclomatic complexity is traditionally less desirable, as it would theoretically have a larger number of edge cases wherein a potential bug could hide.

1.1. Objectives

The objective of this paper is to analyze the relationship between cyclomatic complexity and bug-introducing commits in Python packages on PyPI, using the Sliwezki-Zeller-Zimmermann (SZZ)⁵ algorithm to identify bug-introducing commits. By mining a representative sample of important Python packages, the aim is to understand whether higher complexity functions are more likely to be associated with bugs, and how this relationship has evolved over time.

1.2. Related work

Previous research has explored the relationship between code complexity and software defects in various programming languages⁶⁻⁸, but there is a lack of comprehensive studies focused specifically on Python packages in the PyPI ecosystem. Additionally, while the SZZ algorithm has been widely used to identify bug-introducing commits, its application in the context of Python packages and its relationship with cyclomatic complexity does not appear to have been thoroughly investigated yet.

1.3. Research Question

The paper will seek to answer the following research questions:

1. Can a meaningful correlation be observed between cyclomatic complexity and bug-proneness in Python packages on PyPI?
2. How do bug-fixing commits affect the cyclomatic complexity of functions? Do they tend to increase, decrease, or stay the same?

2. Methods

2.1. Package Selection

Initially, the study aimed to focus on the ten most downloaded packages on PyPI. However, on further investigation, it was found that download count alone was not a reliable indicator of whether the package is truly a “lynchpin” package within the ecosystem. For example, the most downloaded package as of the time of writing was **boto3**, which is exclusively used by Amazon Web Services (AWS). The download count is inflated by the widespread use of AWS services. However, it is not general purpose and is completely irrelevant to the majority of Python developers.

To better capture the intended population of important packages, the selection process was adjusted to consider the top-ranked packages in the Libraries.io ranking of the most depended-upon PyPI projects⁹. This ranking is based on the number of other packages that depend on a given package (reverse dependencies), which is a more direct measure of its centrality and importance within the ecosystem. Starting from the top of that ranking, packages were considered in order until ten projects with resolvable GitHub source repositories had been identified. The selected release version and basic package metadata were then verified against the corresponding PyPI project metadata³.

Package	Version	Commit
requests	2.33.1	04d750509b90
numpy	2.4.4	0f897ea33050
pytest	9.0.3	8f81c76744da
pandas	3.0.2	566284d0ce87
pydantic	2.13.3	14e0f2ba41ea
matplotlib	3.10.9	f4cc437d1bd7
pytest-cov	7.1.0	574f0c59cf89
click	8.3.3	73e155006526
scipy	1.17.1	815ef31f7e66
black	26.3.1	9fd9ea283597

Table 1: The selected packages and their number of analyzed functions.

The final sample spans scientific computing (numpy, scipy, pandas), testing infrastructure (pytest, pytest-cov), validation (pydantic), plotting (matplotlib), command-line tooling (click), code formatting (black), and HTTP infrastructure (requests). For each package, its GitHub repository was cloned, and commits were mined for a five-year window between 05. May 2021 and 05. May 2026. Across these repositories, the analysis covered 47,151 functions in total.

2.2. Complexity Analysis

Many of the selected packages are written in multiple programming languages aside from Python (for example, parts of numpy are written in C and Fortran). Therefore, a cross-language cyclomatic complexity analysis tool was required for a comprehensive assessment. Lizard, a Python module which can compute the cyclomatic complexity of source code in over 25 programming languages¹⁰, was therefore chosen for this purpose.

2.3. SZZ Attribution

Due to the limited scope of this study, Github Issues were not used to identify bug-fixing commits. Instead, bug-fixing commits are identified heuristically by matching commit messages against a set of keywords: *fix*, *fixes*, *fixed*, *bug*, *bugfix*, *regression*, *hotfix*, *patched*, and *defect* (case-insensitive).

To approximate likely bug-introducing commits, the study applies a simplified SZZ procedure⁵. For each identified bug-fixing commit, deleted lines from supported source files are collected from the diff against the parent revision and grouped by the function they belonged to. `git blame --line-porcelain` is then run on those contiguous deleted-line ranges against the parent revision to find which commits originally introduced them. Blamed commits that coincide with the bug-fixing commit itself, or that are null commits, are excluded. The remaining blamed revisions are treated as candidate bug-introducing commits.

To estimate the effect of each bug-introducing commit on function complexity, the function is located in both the version before and after the blamed commit. The primary matching strategy is by exact function name. If the function is only found on one side (for example because it was moved or renamed), a structural fallback is used. Candidate functions of the same kind are scored based on how much their line spans overlap, how similar their names are according to `SequenceMatcher` from Python's `difflib` standard library module, and how far their boundaries have shifted.

A candidate is accepted if the overlap ratio reaches at least 60%, or if there is any overlap at all and the boundary shift is within 6 lines, or if the names are nearly identical and the boundary shift is within 3 lines. If no match can be established on either side, the entry is excluded from the complexity-delta analysis. This is consistent with the broader warning that mining GitHub repositories requires careful treatment of noisy metadata and workflow-specific artifacts¹¹.

Only files with an extension recognized by Lizard were included in the analysis. Documentation files (`.md`, `.rst`, `.txt`, `.asciidoc`, etc.) and anything under a `doc` or `docs` directory were excluded, as were test files (`test*.py`, `*_test.py`), test directories (`test/`, `tests/`, `testing/`), build and environment artifacts (`build/`, `dist/`, `.git/`, etc.), and example and benchmark directories (`example/`, `examples/`, `benchmarks/`, etc.).

3. Results

3.1. Dataset Overview

The final dataset contains 47,151 analyzed functions across ten repositories. Of these functions, 12,882 were touched by at least one bug-fixing commit. Summed over the selected repositories, the mining pass identified 9,034 bug-fix commits and 15,442 unique bug-introducing commits attributed by the simplified SZZ stage. The selected repositories vary substantially in size. At the low end, pytest-cov contributes 52 analyzed functions, while scipy contributes 13,603 (see Table 2).

Rank	Package	Version	Functions analyzed
1	requests	2.33.1	247
2	numpy	2.4.4	7872
3	pytest	9.0.3	1966
4	pandas	3.0.2	7936
5	pydantic	2.13.3	3334
6	matplotlib	3.10.9	11007
7	pytest-cov	7.1.0	52
8	click	8.3.3	507
9	scipy	1.17.1	13603
10	black	26.3.1	627

Table 2: Packages with their reverse dependency rank, version, and number of analyzed functions.

3.2. Complexity and Bug-fix Correlation

To estimate the overall relationship between function complexity and bug-fix activity, two correlation coefficients were computed at the package level. The Pearson coefficient measures the strength of the linear relationship between two variables, ranging from -1 (perfect negative linear relationship) to $+1$ (perfect positive linear relationship), with 0 indicating no linear association¹². The Spearman coefficient is a rank-based alternative that captures monotonic relationships more broadly, without assuming linearity or a particular distribution of the data¹³.

Both are reported here because complexity distributions in real codebases tend to be heavily right-skewed, which makes the rank-based Spearman measure a more robust primary indicator. Both measures are consistently weakly positive across all ten repositories, indicating that higher complexity functions tend to be associated with more bug-fix commits.

The strongest Spearman correlation appears in black ($r_s = 0.429$), followed by pytest ($r_s = 0.319$) and pytest-cov ($r_s = 0.296$), while the weakest is pydantic ($r_s = 0.134$). The Pearson correlation coefficients show a similar pattern (see Table 3).

Package	Spearman r_s	Pearson r
black	0.429	0.548
pytest	0.319	0.456
pytest-cov	0.296	0.499
pandas	0.291	0.407
click	0.289	0.534
matplotlib	0.262	0.348
scipy	0.250	0.207
numpy	0.227	0.256
requests	0.207	0.251
pydantic	0.134	0.188

Table 3: Correlation coefficients between function complexity and bug-fix commit frequency for the ten analyzed packages.

Taken together, these results seem to provide an answer to the first research question: higher function complexity is consistently associated with more bug-fix activity, although the strength of that association differs considerably across projects.

3.3. Maintenance Concentration

Most analyzed functions fall into the 1–20 complexity score bucket, and only 26.5% of them were touched by at least one bug-fix commit. In the 21–40 bucket, that share rises to 54.6%. The 41–60 and 61–80 buckets remain above 52%, and the smaller high-complexity buckets are mostly similar or higher, although those ranges contain far fewer functions. The mean number of bug-fix commits per function (including functions with zero bug-fix commits) follows the same pattern, rising from 0.48 in the 1–20 bucket to 1.76 in the 21–40 bucket and 2.16 in the 41–60 bucket. The drop-off in the highest buckets is likely due to the extremely small sample sizes.

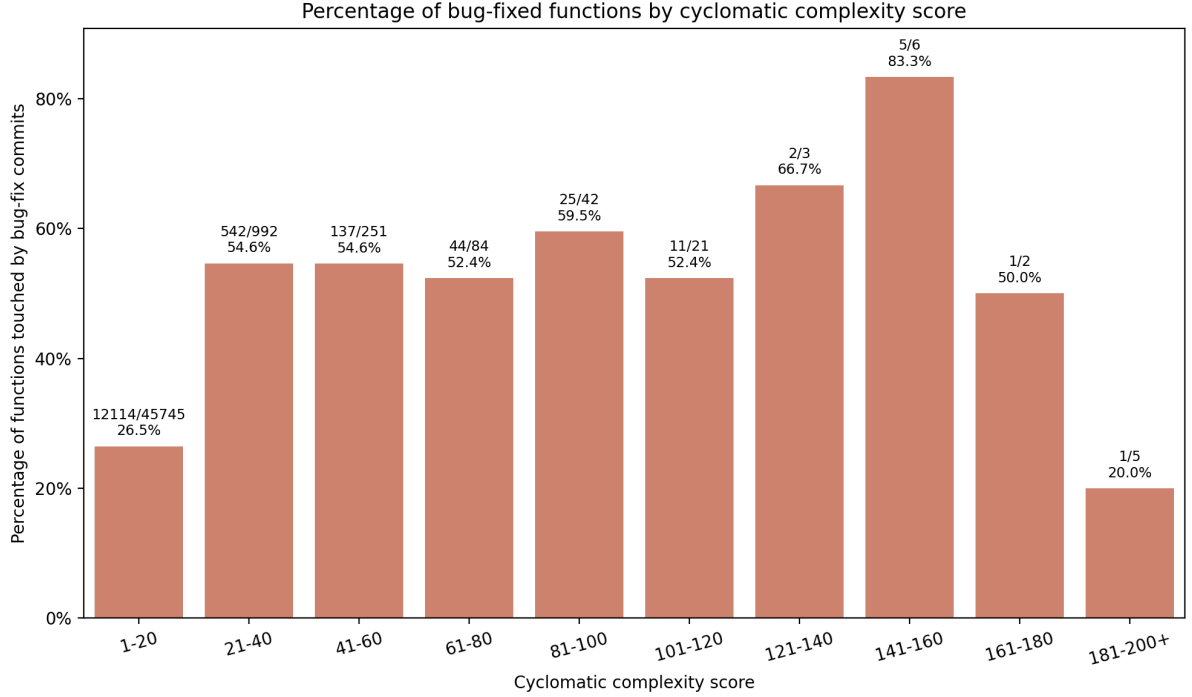


Figure 1: Share of functions touched by at least one bug-fix commit, and mean bug-fix commits per function, bucketed into groups of 20.

Complexity bucket	Functions	Bug-fixed functions	Mean bug-fix commits per function
1-20	45745	12114	0.48
21-40	992	542	1.76
41-60	251	137	2.16
61-80	84	44	1.39
81-100	42	25	1.69
101-120	21	11	0.95
121-140	3	2	1
141-160	6	5	4.17
161-180	2	1	1.5
181-200+	5	1	1.8

Table 4: Complexity bucket summary. The mean bug-fix count is computed over all functions in each bucket, including functions with zero bug-fix commits.

The cumulative concentration plot answers a slightly different question. It orders all functions from highest complexity to lowest complexity, then shows how much of the total bug-fix activity has been accumulated after taking the top $x\%$ of that ordering. If bug-fix activity were spread evenly across the codebase, the curve would follow the diagonal. Instead, it stays above the diagonal throughout. The 1% most complex functions account for 3.8% of all bug-fix commits, the top 5% account for 16.7%, the

top 10% account for 29.1%, the top 20% account for 47.3%, and the top half account for 73.8% (see Figure 2). This means maintenance work is meaningfully concentrated in more complex functions, but not so strongly that a very small minority of functions accounts for nearly all fixes.

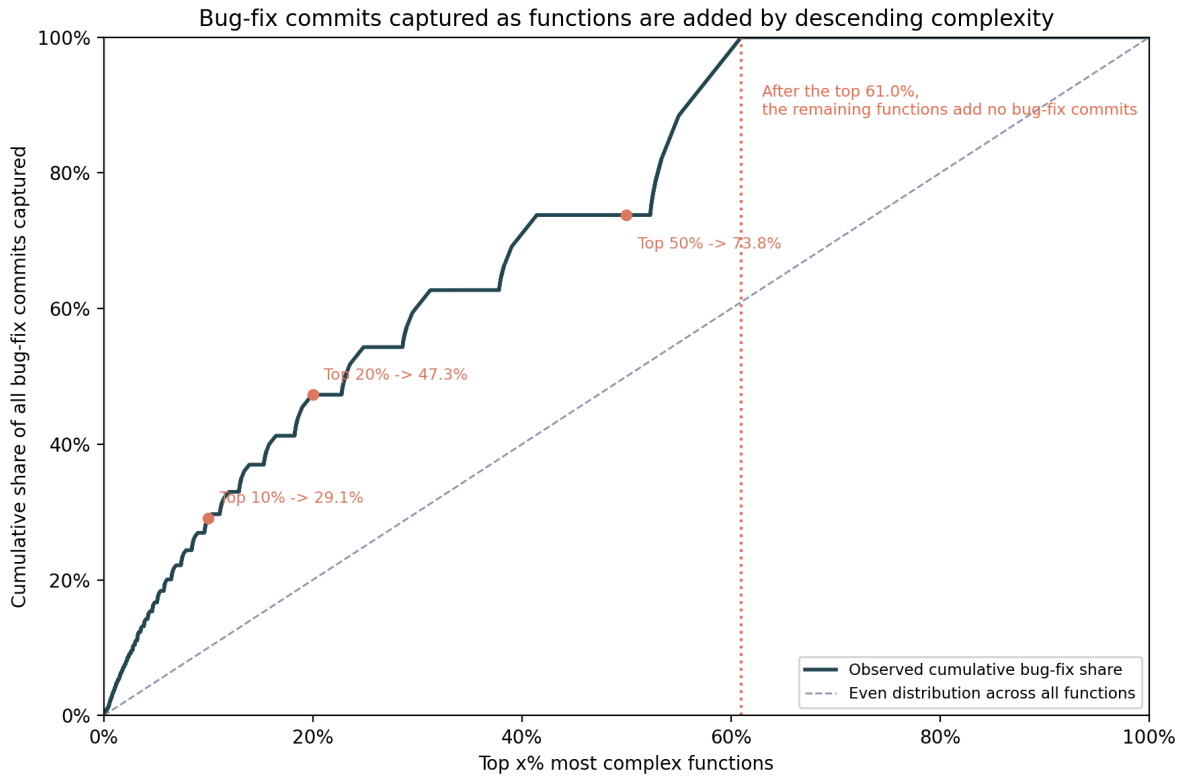


Figure 2: Cumulative concentration of bug-fix activity among the most complex functions.

3.4. Complexity Changes in Bug-fixing Commits

The bug-fix event summary shows that most bug-fixing touches do not substantially alter function complexity. Of all bug-fix events, 62.0% fall into the “No change” category. Complexity increases account for 13.0%, decreases for 5.5%, newly introduced functions for 13.3%, and deleted or renamed functions for 6.1%. In other words, the most common outcome of a bug-fixing commit is not simplification but local repair inside an existing function.

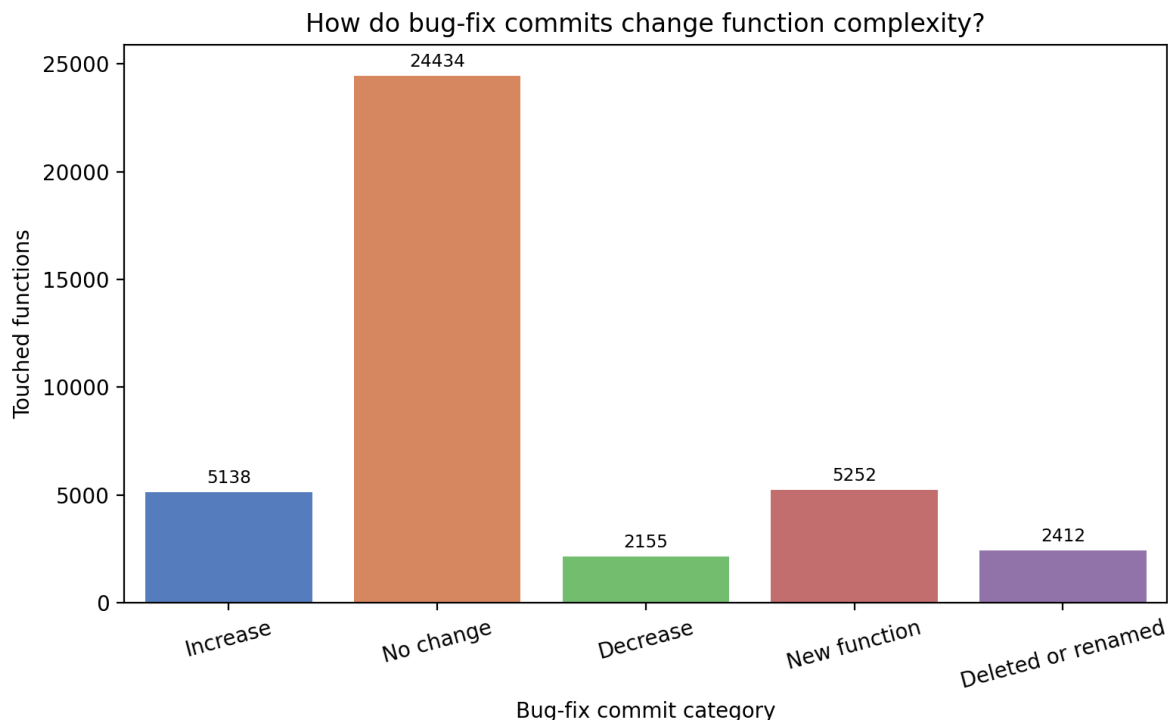


Figure 3: Complexity-change categories observed at bug-fixing commits. No-change outcomes dominate, and increases are more common than decreases.

This distribution suggests that complexity is related more strongly to where bug-fix activity occurs than to a general tendency for fixes to simplify code. When complexity does change, increases are more common than decreases, which indicates that fixes often add guards, branches, or special-case handling rather than removing structural decision points. Many fixes therefore appear to adjust behavior inside already complex functions without materially reducing their measured complexity.

3.5. SZZ Attributions

The SZZ results tell a similar but more qualified story. Across all attributed rows, “No change” remains the largest category at 39.7%, followed by “Function pair not recoverable” at 24.5% and “New function” at 17.9%. Complexity increases account for 12.4% of all SZZ attributions, while decreases account for 5.3% (see Figure 4). The large unrecoverable share is important because it reflects the difficulty of matching historical function pairs conservatively across real repository histories.

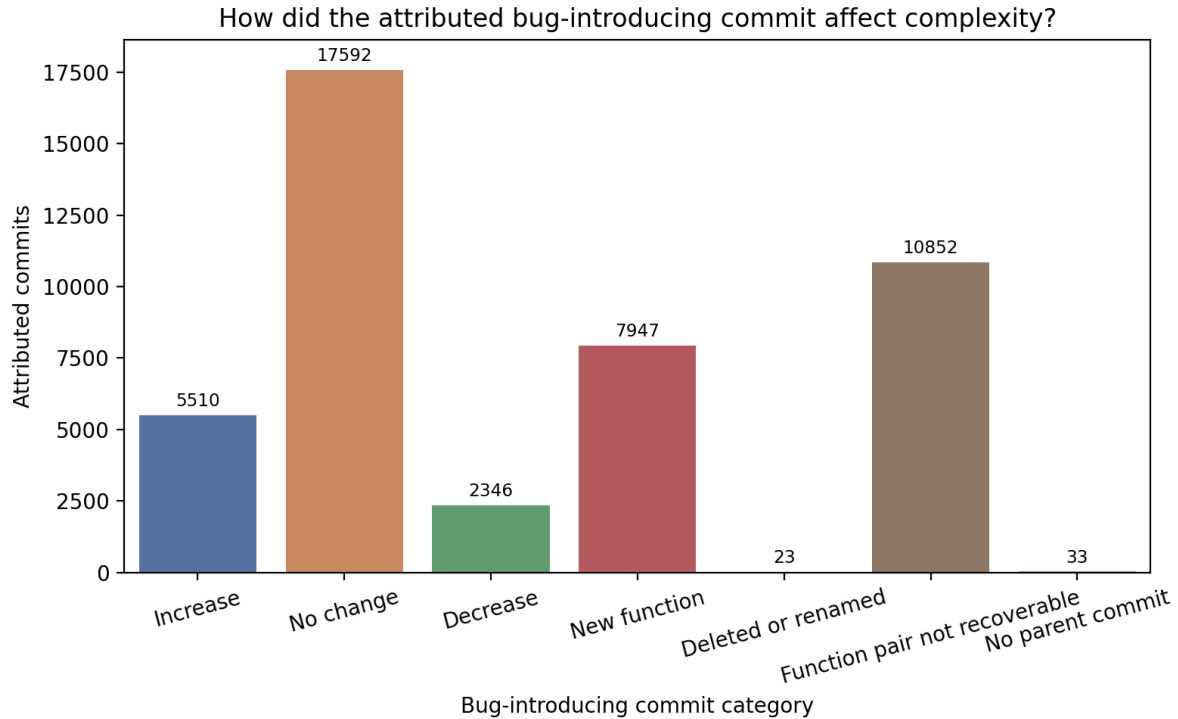


Figure 4: Complexity-change categories for simplified SZZ attributions. A substantial share of attributions cannot be recovered at function level, but matched rows still show more increases than decreases.

When only matched attributions are considered, the pattern becomes clearer: 69.1% of matched SZZ rows show no change, 21.7% show an increase, and 9.2% show a decrease. Thus, among attributable function pairs, increases are distinctly more common than decreases, but unchanged functions still dominate. This suggests that bug-introducing commits are more likely to add complexity than remove it, although a rise in cyclomatic complexity is neither necessary nor sufficient as evidence for a later defect.

3.6. Time To Fix

The temporal distance between attributed bug-introducing and bug-fixing commits is also substantial. Across 27,526 unique attributed commit pairs with valid, non-negative timestamps, the median lag is 751.1 days, with an interquartile range from 189.0 to 1807.8 days. The package-level summaries show substantial variation. Pydantic has the shortest median lag at 133.1 days, while pytest-cov reaches 2332.5 days, requests 1888.4 days, and scipy 1232.6 days. Many attributed bugs therefore remain latent for months or years before being repaired.

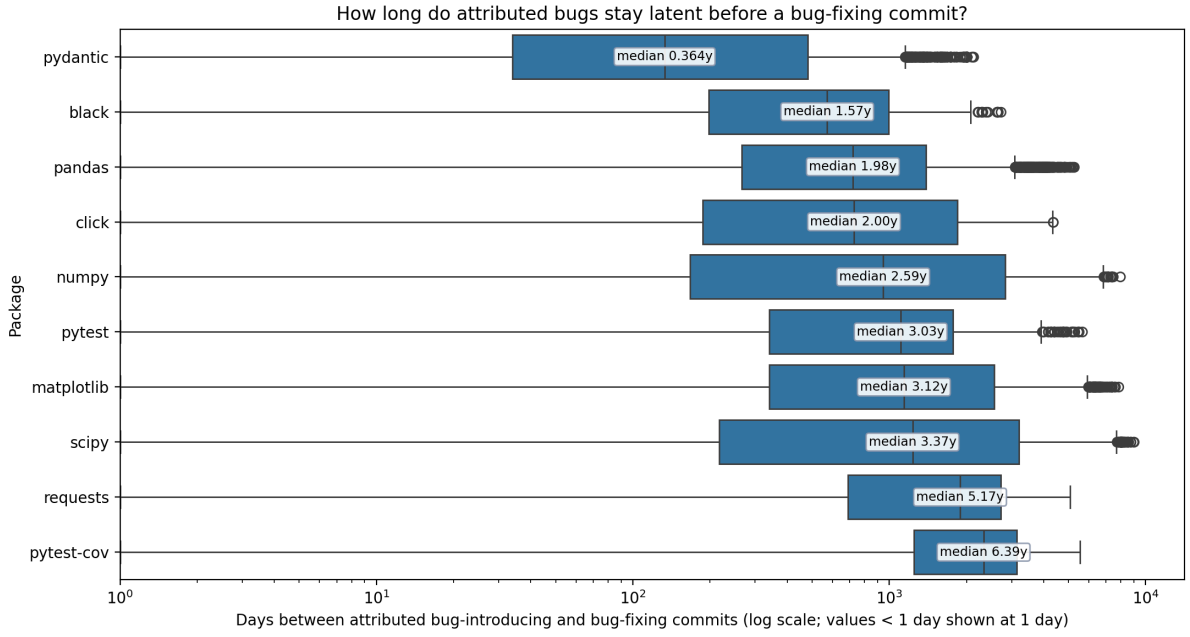


Figure 5: Distribution of time lags between attributed bug-introducing commits and their corresponding fixes.

Overall, these findings answer the second research question in a qualified way. Bug-fixing commits usually leave measured complexity unchanged, and when complexity does change, it tends to increase rather than decrease. The simplified SZZ results point in the same direction for attributed bug-introducing commits, but they also show that a substantial fraction of historical function pairs cannot be recovered confidently. The overall picture is therefore not one of direct causation, but of elevated maintenance risk: higher complexity is associated with more bug-fix activity and a larger share of repeated maintenance, while the actual path from change to defect remains shaped by package-specific workflows and long time lags.

4. Discussion

The results show a consistent relationship between cyclomatic complexity and later maintenance, but they do not support a simple claim that complex code is automatically defective. Across all ten packages, the correlation between complexity and bug-fix frequency is always positive, and the concentration analyses show that more complex functions absorb a disproportionate share of bug-fix activity. At the same time, the effect is moderate rather than overwhelming. The top 10% most complex functions account for 29.1% of bug-fix commits, and the top 20% account for 47.3%. Complexity therefore looks like a meaningful risk signal, but not a stand-alone explanation for why bugs occur.

The bug-fix and SZZ change summaries also point to a more specific interpretation. Most bug-fixing commits do not reduce measured complexity, and most matched bug-introducing commits also show no complexity change. When changes do occur, increases are more common than decreases in both views. This suggests that complexity matters less as a one-time threshold crossed by a single commit and more as a structural condition that can make a function harder to reason about, harder to test thoroughly, and more likely to accumulate compensating logic over time. In practical terms, the findings fit a maintenance-risk interpretation better than a direct causal interpretation.

The long lag between attributed bug-introducing and bug-fixing commits reinforces that point. The median lag across valid attributed pairs is just over two years, and several packages show much longer medians. That makes it difficult to treat present complexity as a direct explanation of a later failure. A function may become buggy under one local context and only fail much later after surrounding code, inputs, or usage patterns change. Complexity appears to increase the opportunity for such failures to persist, but the data do not support the stronger claim that it alone produces them.

4.1. Limitations

Several methodological limits constrain how strongly these results should be generalized. Bug-fixing commits were identified through a commit-message heuristic rather than issue links or machine learning-based labelling. This likely introduces both false positives and false negatives. Some genuine fixes will not use the selected keywords, while some matching commits may concern refactoring, documentation, or maintenance work that is not truly bug fixing.

The simplified SZZ procedure is also deliberately conservative. A substantial share of attributed rows could not be recovered as function pairs, which reflects the difficulty of tracing historical changes through renames, refactorings, moves, and partial rewrites. This was deemed preferable to overly aggressive matching, but it means the function-

level SZZ results likely understate the number of attributable pairs and may be biased toward functions whose identity remains easier to preserve across revisions.

Another limitation is the use of cyclomatic complexity as the sole structural metric. Cyclomatic complexity is well established and easy to compute across languages, which makes it suitable for a multi-language repository sample, but it does not capture all forms of difficulty relevant to maintenance. For example, a switch statement with many cases may have high cyclomatic complexity but still be straightforward to understand, while a recursive function can have low complexity and still be hard to parse.

The package sample is also intentionally narrow. The ten selected projects are important packages by reverse dependency rank, but they are not representative of the entire Python ecosystem. The sample includes very large scientific packages and much smaller infrastructure packages, and those scale differences can affect both the stability of per-package correlations and the shape of the aggregate results. The findings should therefore be read as evidence about central, high-impact packages rather than about all Python projects.

Finally, the analysis focuses on production source files and explicitly excludes tests, examples, and benchmarks. While that serves to reduce noise, it also means the study does not capture defects that are concentrated in test scaffolding or developer tooling. The results therefore describe the relationship between complexity and maintenance purely in production code.

5. Conclusion

Correlation does not imply causation

— F.A.D (Pseudonymous Author)¹⁴

This study examined whether cyclomatic complexity is associated with bug-proneness in a sample of important Python packages and how bug-fixing activity relates to complexity change. The answer to the first research question is yes, but only in a qualified sense. Across all ten analyzed repositories, both Pearson and Spearman correlations between function complexity and bug-fix activity are positive, and the concentration analyses show that more complex functions carry a disproportionate share of maintenance.

The answer to the second research question is more nuanced. Most bug-fixing commits do not materially change measured complexity, and the same is true for most matched SZZ attributions. When complexity does change, increases are more common than decreases. Bug fixes therefore do not usually simplify the affected code, and bug-introducing commits are more likely to add structural decision points than remove them. At the same time, the substantial share of unrecoverable SZZ pairs and the long median lag between attributed introduction and later repair show that defects can emerge through longer historical processes than a single complexity increase can explain.

Overall, the findings support a pragmatic conclusion. Higher cyclomatic complexity is not a direct proof that a function is faulty, but it is a useful warning sign for where defects and repeated maintenance are more likely to accumulate. For maintainers of high-impact Python packages, complexity appears to be most valuable as a prioritization signal for review, testing, and refactoring, rather than as a causal diagnosis on its own.

5.1. Future Work

There are a few notable avenues for future research.

1. Replace the commit-message heuristic with issue-linked or ML classifier-based bug-fix identification.
2. Extend the package sample beyond just the few most depended-upon projects to test how strongly the findings generalize.
3. Compare cyclomatic complexity with additional complexity measures such as cognitive complexity.
4. Use a more intelligent SZZ tracing procedure that can recover more function pairs and better handle merges, renames, and refactorings.

References

1. Van Rossum G, Drake FL. *Python 3 Reference Manual*. CreateSpace; 2009.
2. Chaudhary P, Agrawal L, Ali A. Modern programming languages—characteristics and recommendations for instruction. *Issues in Information Systems*. 2025;26(2).
3. Python Software Foundation. Python Package Index (PyPI). Accessed May 4, 2026. <https://pypi.org/>
4. McCabe TJ. A Complexity Measure. *IEEE Transactions on Software Engineering*. 1976;(4):308-320. doi:10.1109/TSE.1976.233837
5. Sliwerski J, Zimmermann T, Zeller A. When Do Changes Induce Fixes?. In: *Proceedings of the 2005 International Workshop on Mining Software Repositories*. 2005:1-5. doi:10.1145/1083142.1083147
6. Bachmann A, Bird C, Rahman F, Devanbu P, Bernstein A. The Missing Links: Bugs and Bug-Fix Commits. In: *Proceedings of the Eighteenth ACM SIGSOFT International Symposium on Foundations of Software Engineering*. 2010:97-106. doi:10.1145/1882291.1882308
7. Hassan AE. Predicting Faults Using the Complexity of Code Changes. In: *2009 IEEE 31st International Conference on Software Engineering*. 2009:78-88. doi:10.1109/ICSE.2009.5070510
8. Nagappan N, Ball T. Use of Relative Code Churn Measures to Predict System Defect Density. In: *Proceedings of the 27th International Conference on Software Engineering*. 2005:284-292. doi:10.1109/ICSE.2005.1553571
9. SonarSource Sàrl. Libraries.io. Accessed May 4, 2026. <https://libraries.io/pypi>
10. Yin T. Lizard: An extensible Cyclomatic Complexity Analyzer. *Astrophysics Source Code Library*. 2019;(1906):11. Accessed May 14, 2026. <https://pypi.org/project/lizard/>
11. Kalliamvakou E, Gousios G, Blincoe K, Singer L, German DM, Damian D. An In-Depth Study of the Promises and Perils of Mining GitHub. *Empirical Software Engineering*. 2016;21(5):2035-2071. doi:10.1007/s10664-015-9393-5
12. Pearson K. VII. Note on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London*. 1895;58(347–352):240-242. doi:10.1098/rspl.1895.0041
13. Spearman C. The Proof and Measurement of Association between Two Things. *The American Journal of Psychology*. 1904;15(1):72-101. doi:10.2307/1412159
14. F.A.D. Biology as an "Exact" Science. *Nature*. 1900;62(1594):50.

AI Declaration

The usage of AI tools within this project is documented here. I declare that I have documented all interactions with AI tools, including the prompts used and the outputs received.

System	Prompt	Usage
GitHub Copilot 1	I want you to rewrite my template from LaTeX to Typst, and make it look like it did before.	Recreated the template in Typst, maintaining the original appearance.
GitHub Copilot 2	In Code/, write a Python script that finds the top most depended-upon Python packages and analyzes the correlation between function cyclomatic complexity and bug-fix frequency over an adjustable time window, with plots and tables.	Created the analysis pipeline.
GitHub Copilot 3	Use pip to install any required packages, not conda.	Added pip dependency setup.
GitHub Copilot 4	Correct the package discovery so the ranking reflects dependency counts and the analysis uses real source repositories rather than release repositories.	Fixed discovery and repo resolution.
GitHub Copilot 5	Make the analysis practical to run by adding adjustable bounds for the mined recent commit history.	Added bounded commit mining.
GitHub Copilot 6	Use ProcessPoolExecutor to accelerate the script.	Parallelized package mining.
GitHub Copilot 7	Add compact logging and terminal progress bars, show phases such as cloning, mining, SZZ, and plotting, and remove Typst table generation.	Added logs and progress bars; removed Typst exports.
GitHub Copilot 8	Fix the parser warning noise produced while mining repositories.	Suppressed parser warnings.
GitHub Copilot 9	Fix the Windows UnicodeDecodeError that occurs while reading subprocess output during mining.	Hardened subprocess decoding.

System	Prompt	Usage
GitHub Copilot 10	Implement flexible caching so repeated runs can reuse already mined information while still respecting parameter changes.	Added parameter-aware mining cache.
GitHub Copilot 11	Add analysis support for native code too and update the progress bars more often.	Added mixed-language analysis; increased progress updates.
GitHub Copilot 12	Use all languages recognized by lizard instead of restricting the non-Python analysis to C-family files.	Expanded lizard language coverage.
GitHub Copilot 13	Add a CLI option to restrict the analysis to Python files only while keeping mixed-language analysis as the default.	Added Python-only analysis mode.
GitHub Copilot 14	Refactor the analysis script into helper modules and keep the main script as the orchestration entrypoint.	Reorganized the analysis code into helper modules.
GitHub Copilot 15	Replace the PyDriller repository mining implementation with direct Git CLI queries and keep the SZZ analysis working on top of that path.	Replaced PyDriller-based mining with Git CLI mining and SZZ integration.
GitHub Copilot 16	Improve the SZZ output by separating unresolved cases and adding rename-aware matching for moved files and renamed functions.	Added SZZ attribution categories and rename-aware matching.
GitHub Copilot 17	Rename the stale non-Python parser helper so it reflects the current lizard-based multi-language behavior.	Renamed the non-Python parser helper for clarity.
GitHub Copilot 18	Add NumPy-style docstrings to the important analysis, mining, and reporting functions.	Added NumPy-style docstrings to key functions.
GitHub Copilot 19	Rewrite the repository README so it reads like a cohesive project README rather than patchwork.	Reworked the README structure and usage documentation.
GitHub Copilot 20	Make plotting reusable from precomputed results, add richer reporting outputs, and export grouped raw JSON.	Added plot-only rendering, extra summaries and plots, and raw JSON output.

System	Prompt	Usage
GitHub Copilot 21	Persist commit dates for bug-fixing and bug-introducing commits and add a time-series plot for bug-fix commits.	Added commit-date export and bug-fix timeline reporting.
GitHub Copilot 22	Replace the many CLI flags with a TOML config file, keep only <code>-cfg</code> in the CLI, and make rename-matching thresholds configurable.	Introduced TOML-based configuration and configurable rename matching.
GitHub Copilot 23	Add config validation, timeline granularity options, and clearer config comments.	Added stricter config validation and documented config options.
GitHub Copilot 24	Replace the less useful plot, improve the reporting plots, and make the bucket ranges clearer.	Reworked plots and simplified the complexity buckets.
GitHub Copilot 25	Preserve full commit messages in raw JSON and regenerate compact CSV messages from that source.	Made raw JSON the source of truth for compact message exports.
GitHub Copilot 26	Exclude documentation files from the complexity and SZZ analysis.	Excluded documentation files from analysis inputs.
GitHub Copilot 27	Replace the old package-selection source with Libraries.io and update the supporting documentation accordingly.	Switched package discovery to Libraries.io and updated the related documentation.
GitHub Copilot 28	Look at my code. See if I missed something in the SZZ Attribution section in my paper.	Expanded the SZZ Attribution section to include more details on the matching strategy and the handling of unmatched cases.
GitHub Copilot 29	Read the rest of my paper and draft me an abstract in English and German.	Drafted an abstract in English and German based on the paper's content.
GitHub Copilot 30	Add a CSV table to the Complexity Bucket section showing the mean bug-fix commits per function.	Added a CSV table to the Complexity Bucket section in 03_Results.typ.

Declaration of Authorship

I hereby confirm that I have personally and independently prepared the present work and have not used any sources or aids other than those specified. All passages taken verbatim or in substance from other sources are identified as such. The drawings, illustrations and tables in this work are created by me or have been provided with an appropriate source reference. This work has not been submitted by me to any other university in the same or similar form for the acquisition of an academic degree.

Frankfurt, 14.05.2026

Rubin Chempananickal James